

TX TIMEOUT과 체인 재편성(REORG)

복구 전략 원리 · Nonce · Gas Bump · MINED vs CONFIRMED vs FINALIZED

M3 TX 상태머신

S19 이론

Day 05

아젠다 · 학습 목표

M3 · SESSION 19 · 강의 55분 전체 이론

강의 구성 (55분 이론)

- § 1 TX TIMEOUT — 원인과 구조
- § 2 Gas Bump — RBF 전략
- § 3 REORG 발생 원리
- § 4 MINED / CONFIRMED / FINALIZED 구분
- § 5 3중 복구 전략 비교 정리

학습 목표

- TX TIMEOUT / REORG 원인을 구조적으로 설명할 수 있다
- CONFIRMED와 FINALIZED의 차이와 각각의 활용 기준을 말할 수 있다
- REVERT / TIMEOUT / REORG 3중 복구 전략을 구분하여 서술할 수 있다
- reorgedReq가 왜 필요한지 설명할 수 있다

선수 지식: S17 TX 상태머신 / S18 REVERT 처리 — 상태 전이표(VALID_TRANSITIONS) 이해 필요

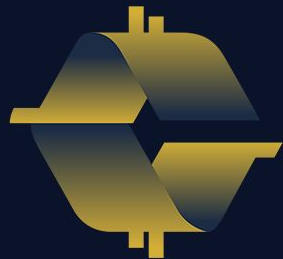
01

TX TIMEOUT

TX가 mempool에 갇히는 이유
가스비 경쟁과 Nonce 블로킹 구조

§ 1

TIMEOUT



COINCRAFT
ACADEMY

TX가 mempool에 갇히는 이유

§ 1 TX TIMEOUT · 가스비 경쟁 원리

- › 채굴자(검증자)는 **가스비가 높은 TX**부터 블록에 포함시킨다
- › 가스비가 낮은 TX는 mempool에서 **무기한 대기** 상태가 된다
- › 네트워크 혼잡 시 가스비 기준이 급등 → 이전에는 적정했던 TX가 **stuck**
- › 결과: PENDING 상태 무기한 유지 → **타임아웃 감지 로직** 필요

주의: 가스비를 지나치게 낮게 설정한 TX는 네트워크에서 영원히 처리되지 않을 수 있다. mempool 잔류 TX는 노드 재시작 시 폐기되기도 한다.

Nonce와 큐 블로킹

§ 1 TX TIMEOUT · Nonce 순서 강제와 연쇄 블로킹

- › 이더리움은 주소별 **Nonce** 순서를 강제한다 — 반드시 순서대로 처리
- › Nonce 5가 stuck이면 → Nonce 6, 7, 8... 전부 **블로킹** 상태

정상 처리 흐름

- Nonce 0 → 채굴됨
- Nonce 1 → 채굴됨
- Nonce 2 → 채굴됨
- Nonce 3 → 채굴됨

Nonce 갭 발생 시

- Nonce 0, 1 → 채굴됨
- Nonce 2 → **stuck** (가스비 부족)
- Nonce 3, 4, 5 → 전부 대기 중
- Nonce 2가 해결될 때까지 진행 불가

핵심: Nonce 갭 단 하나가 이후 모든 TX를 막는다. gas bump는 stuck TX의 Nonce를 재사용해야 한다.

02

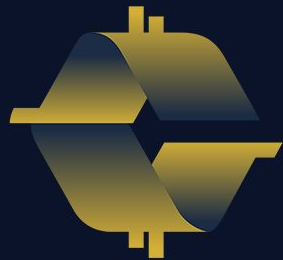
Gas Bump 전략

Replace-by-Fee(RBF) 원리

handleTimeout 흐름 분석

§ 2

RBF



COINCRAFT
ACADEMY

Replace-by-Fee (RBF)

§ 2 Gas Bump · 동일 Nonce 재전송으로 기존 TX 대체

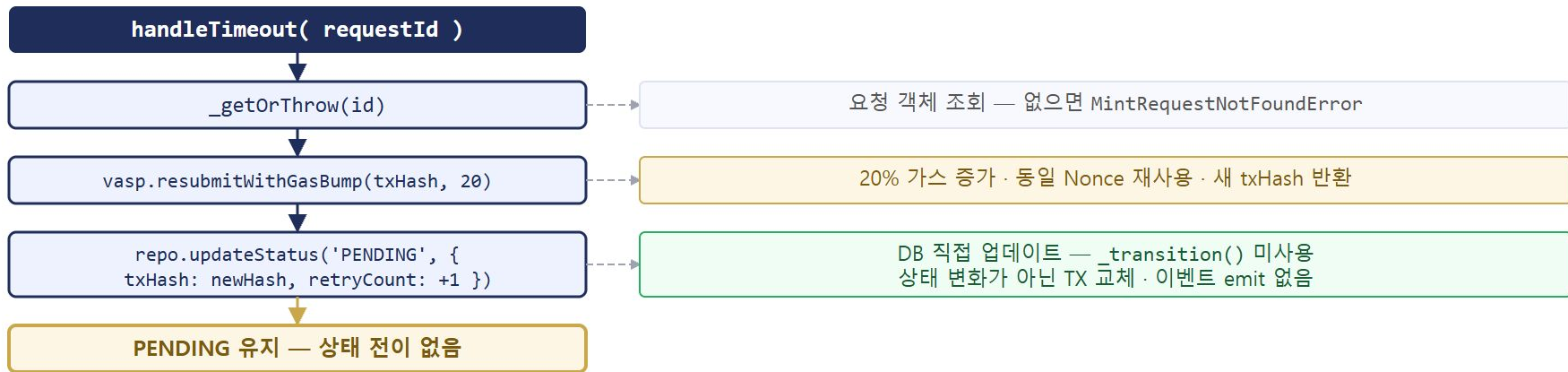
- › 동일 Nonce로 가스비를 높여 재전송하면 기존 TX가 자동 대체된다
- › 조건: 새 가스비 $\geq$ 기존 가스비 $\times 1.1$ (대부분의 노드 정책)
- › 우리 구현: 1.2배 (20% 증가) — 노드 허용 최솟값보다 충분한 여유
- › 결과: 원래 TX는 자동 폐기, 새 TX가 mempool 채굴 우선도 상승

RBF는 stuck TX를 'cancel'하는 표준 방법이다. 같은 Nonce를 재사용하므로 큐 블로킹도 함께 해소된다.

주의: 가스비를 무한정 올리면 비용 폭증. 재시도 횟수(retryCount) 상한 및 최대 가스비 한도 설정 필요.

handleTimeout 흐름

§ 2 Gas Bump · 상태 전이 없는 TX 교체 패턴



`_transition()` 미사용 이유: PENDING→PENDING은 VALID_TRANSITIONS에 정의되지 않음. 상태 변화가 아니라 TX 교체가므로 `repo.updateStatus()` 직접 호출.

이벤트 emit 없음: 외부 원장에 반영할 상태 변화가 없다. txHash만 갱신.

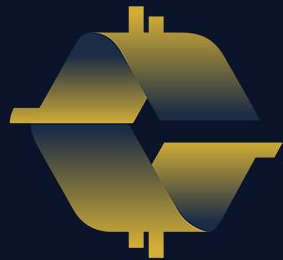
03

REORG 발생 원리

블록 재편성이란 무엇인가
MINED가 갑자기 사라지는 이유

§ 3

REORG



COINCRAFT
ACADEMY

블록 재편성(REORG)이란

§ 3 REORG 원리 · 체인 분기와 정식 채택

- › PoS 네트워크에서도 **동시에 두 블록이 제안**될 수 있다 → 일시적 fork
- › 체인이 분기되면 노드들은 **더 긴(더 무거운) 쪽**을 정식 체인으로 채택
- › 폐기된 쪽 블록의 TX는 **mempool로 복귀**하거나 소실될 수 있다
- › **짧은 REORG**: 1~2블록, 네트워크에서 비교적 자주 발생
- › **깊은 REORG**: 드물지만 가능 — MINED 상태였던 TX가 갑자기 사라짐

MINED ≠ 확정: 블록에 포함됐다는 것은 "현재 정식 체인에 있다"는 의미일 뿐이다. REORG가 발생하면 MINED 상태도 무효가 될 수 있다.

MINED / CONFIRMED / FINALIZED 구분

§ 3 REORG 원리 · 확정 단계별 의미와 REORG 가능성

상태	의미	REORG 가능	원장 업데이트
MINED	블록에 포함됨 (정식 체인 기준)	가능	✗ 미실행
CONFIRMED	충분한 후속 블록 확인 (N개)	매우 낮음	✓ 실행
FINALIZED	2/3+ 검증자 동의 (~12분, Casper FFG)	✗ 불가	✓ 최종 확정

설계 원칙: CONFIRMED에서 원장 업데이트를 수행한다. FINALIZED는 절대 불변의 종단 상태 — 감사 기준으로 활용.

왜 FINALIZED를 기다려야 하는가

§ 3 REORG 원리 · 금융 서비스에서의 확정 기준 선택

CONFIRMED (N블록 기준)

- 실용적 안전 기준 — 대부분의 거래소 채택
- ETH 기준 6~12 confirmation 후 입금 처리
- 빠른 처리 가능 (수분 이내)
- 극히 낮은 REORG 위험 (깊은 REORG는 매우 드물)

FINALIZED (Casper FFG)

- PoS Ethereum 기준 약 **12분 (2 epoch)**
- 2/3 이상 검증자 동의 → 절대 불변
- 느리지만 완벽한 안전성 보장
- 고액 결제·감사 기준으로 활용

교보 시스템 기준: CONFIRMED에서 원장 업데이트 실행 / FINALIZED는 감사 추적 및 최종 정산 기준으로 활용

04

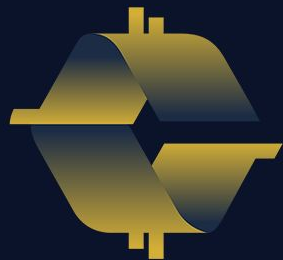
REORGED 전이 설계

handleReorg 흐름

reorgedReq가 필요한 이유

§ 4

REORGED



COINCRAFT
ACADEMY

handleReorg 흐름

§ 4 REORGED 전이 설계 · 대기 후 재확인 패턴



핵심: `reorgedReq` — `_transition()` 두 번째 호출 시 전달하는 객체는 DB에서 새로 조회한, `status`가 `REORGED` 인 객체여야 한다.

reorgedReq가 왜 필요한가

§ 4 REORGED 전이 설계 · VALID_TRANSITIONS 충돌 방지

- > 원본 req 를 그대로 사용 시: `req.status === 'MINED'` → MINED→MINED 전이 시도
- > `VALID_TRANSITIONS['MINED']` 에는 `'MINED'` 가 없음 → **InvalidStatusTransitionError** 발생
- > 해결: `reorgedReq = { ...req, status: 'REORGED' }` — DB에서 갱신된 상태 반영 후 전달
- > `VALID_TRANSITIONS['REORGED']` 에는 `'MINED'` 와 `'FAILED'` 둘 다 정의되어 있음 → 정상 전이 가능

버그 패턴: handleReorg에서 원본 req를 그대로 넘기면 REORG 복구 시도 자체가 에러로 실패한다. `_transition()`에 넘기는 객체의 status는 항상 현재 DB 상태여야 한다.

CONFIRMED → REORGED 누락

§ 4 현재 코드의 결함 · 상태 전이 불완전

결함: `VALID_TRANSITIONS['CONFIRMED'] = ['FINALIZED']`

CONFIRMED 이후 REORG 발생 시 → `_transition()` 즉시 **InvalidStatusTransitionError**. 장부에 이미 반영된 credit은 그대로, 상태는 기록 불가.

항목	현재	수정 후
<code>VALID_TRANSITIONS['CONFIRMED']</code>	<code>['FINALIZED']</code>	<code>['FINALIZED', 'REORGED']</code>
handleReorg 허용 진입 상태	MINED만	MINED + CONFIRMED
CONFIRMED 진입 시 추가 처리	없음 (에러 throw)	<code>reverseMintCredit()</code> → 재제출

MINED vs CONFIRMED 진입의 차이: MINED → 장부 반영 전 → 재제출만. CONFIRMED → 이미 credit 실행됨 → 역분개 먼저, 재제출 나중.

사용자 롤백 통보 전략

§ 4 역분개 이후 · `notifyUserCreditReversed()` 설계 결정

사용자는 이미 "토큰 지급 완료" 알림을 받은 상태다. 역분개(`reverseMintCredit`)는 원장 처리일 뿐 — 사용자 입장에서는 아무 일도 안 일어난 것처럼 보인다.

결정 포인트	즉시 통보	재시도 결과 후 통보
통보 타이밍	역분개 직후 발송 — 사용자 인지 빠름	재시도 성공 시 통보 생략 가능 — UX 간결
재시도 성공 시	"취소" + "재지급" 두 건 발송	성공이면 사용자 입장 변화 없음 — 무통보
재시도 실패(FAILED) 시	운영자 수동 개입 vs 자동 보상 플로우 — 비즈니스 정책 결정 필요	

`notifyUserCreditReversed()` stub이 코드에 있다. 통보 타이밍과 채널(push / 이메일 / 인앱)은 이 메서드 내부 구현에서 결정한다.

권장 전략 — 2단계 통보

§ 4 사용자는 결과만 본다 · REORG는 인프라 내부 사건

REORG를 사용자에게 직접 전달하지 않는다. 역분개 후 재제출까지의 윈도우는 수십 초 — 최종 결과가 나올 때까지 기다렸다가 단일 통보한다.

단계	시점	사용자 노출	내부 처리
Phase 1	reverseMintCredit 직후	잔고 처리 중 표시 — push 없음	내부 알림만
Phase 2-A	재제출 → CONFIRMED	"토큰 지급 완료" 단일 통보	REORG 언급 없음
Phase 2-B	재제출 → FAILED 확정	"지급 실패, 곧 연락드리겠습니다"	보상 플로우 트리거

가상자산이용자보호법 2024.07 시행

역분개 · 지급 실패 통보
명시적 법적 의무 없음

DAXA 모범사례

이용자 통보 권고 — 자율 규제, 법적 강제 아님

출금 차단 시 — 제11조 ★
사전 이용자 통지 + FSC 즉시 보고 의무

거래 기록 보관
실패 TX 포함 **15년 보관** 의무

05

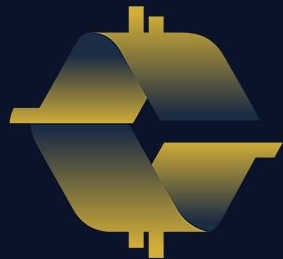
3종 복구 전략 정리

REVERT / TIMEOUT / REORG

장애 유형별 대응 비교

§ 5

비교 정리



COINCRAFT
ACADEMY

3종 복구 전략 비교

§ 5 복구 전략 정리 · 장애 유형별 감지 · 대응 · 최종 상태

장애 유형	감지 방법	즉각 대응	대기	최종 상태
REVERT	VASP webhook	즉시 FAILED + failReason 기록	없음	FAILED
TIMEOUT	폴링 (N분 이상 PENDING)	gas bump 20% 재전송	없음	PENDING → MINED
REORG	VASP webhook	REORGED 상태 전이	5블록 (~60s)	MINED 또는 FAILED

핵심 구분: 세 가지 모두 "TX가 처리 안 됐다"처럼 보이지만 원인과 대응이 완전히 다르다. 원인 진단 없이 재전송하면 이중 처리 위험이 생긴다.

핵심 정리

§ 5 복구 전략 정리 · S19 이론 마무리

- > **TIMEOUT**: 가스비 경쟁에서 진 것 → gas bump (1.2×) 재전송 → 동일 Nonce 재사용
- > **Nonce 갭**: stuck TX 하나가 이후 TX 전체 블로킹 → gas bump로 stuck TX 먼저 해소
- > **REORG**: 블록 재편으로 TX 소실 가능 → REORGED 전이 후 5블록 대기, 재확인
- > **MINED ≠ 확정**: CONFIRMED(N블록) → 원장 업데이트 / FINALIZED(~12분) → 절대 불변
- > **reorgedReq**: `_transition()`에 넘길 객체는 반드시 현재 DB 상태(REORGED)를 반영해야 함
- > **설계 개선 과제**: `VALID_TRANSITIONS['CONFIRMED']`에 `'REORGED'` 추가 + `handleReorg`에 역분개 분기 구현 필요

다음 세션 **S20**: `handleTimeout`, `handleReorg`를 직접 코드로 구현하는 실습 세션. 오늘 이론을 코드로 옮긴다.